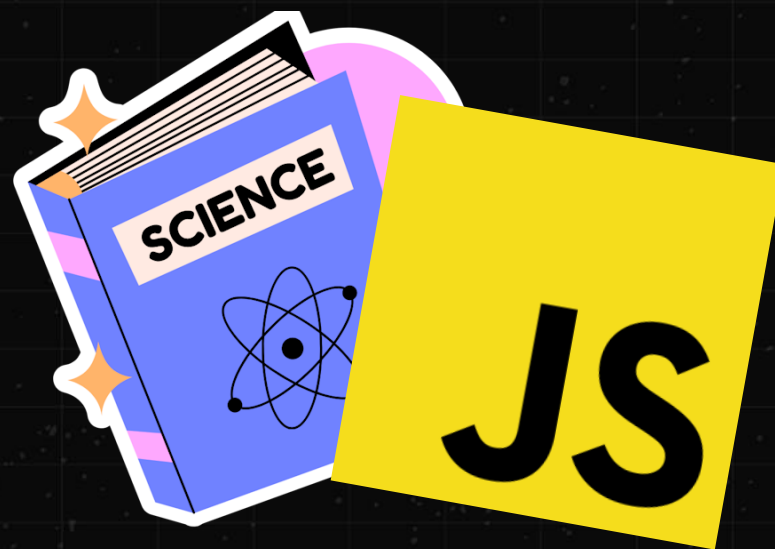


ЛЕКЦИЯ #12

CS BO FRONTEND

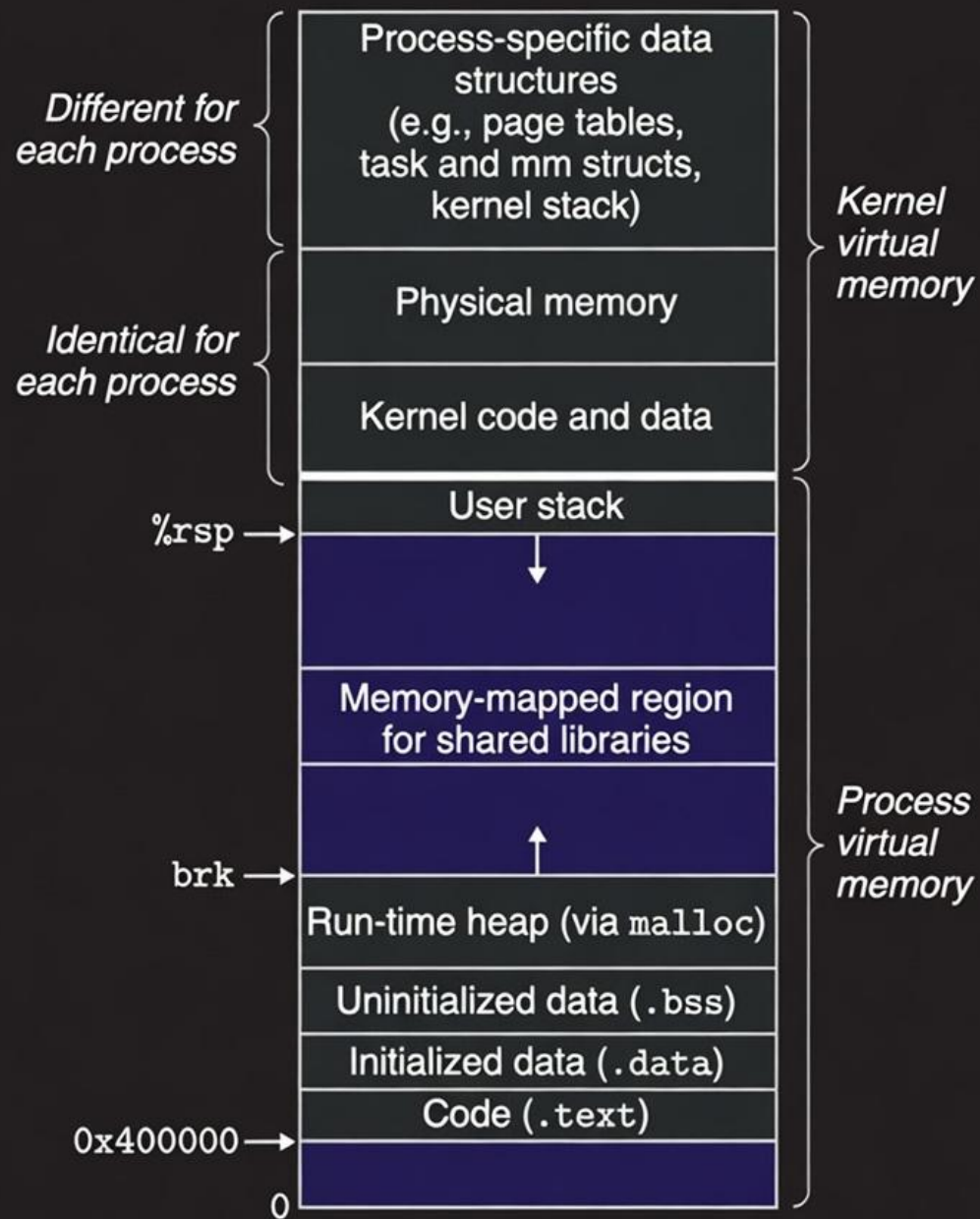


ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Поговорим о многозадачности и роли операционной системы
- * Узнаем, зачем нужна виртуальная память
- * Что такое своп и почему его нужно избегать?
- * Как устроено управление памятью в разных языках
- * И как оно устроено в JS

Figure 9.26

The virtual memory of a Linux process.



ЧТО ОСОБЕННОГО В ЭТОЙ КАРТИНКЕ

- * На ней есть **только одна исполняемая программа** и операционная система
- * Но в **реальности на нашем компьютере «одновременно»** работают десятки и даже сотни программ
- * Возникает вопрос: как организовать память таким образом, **чтобы программы никогда не «пересекались» в памяти**
- * Ведь если такое случится, **это приведёт к UB** со страшными последствиями

ЭТО ОДНА ИЗ ВАЖНЕЙШИХ ЗАДАЧ ОС

- * ОС реализует многозадачность – при этом сами программы могут даже не догадываться об этом
- * ОС создает иллюзию для программы, что она владеет всеми ресурсами на компьютере
- * Достигается это с помощью концепций виртуальной памяти, системы процессов и планировщика задач

ЧТО ЭТО ВСЕ
ЗНАЧИТ?



НАЧНЕМ С ПРОЦЕССОВ

- * Любая **современная многозадачная ОС** использует систему процессов
- * Процесс представляет собой **единицу исполнения любой программы**
- * Процесс состоит **минимум из одного потока исполнения** (инструкции нашей программы) и набора ресурсов
- * **Ресурсами являются адресное пространство памяти, файловые дескрипторы и другое**

КАЖДАЯ ПРОГРАММА ЗАПУСКАЕТ ПРОЦЕСС

- * При этом если вы запустите два раза одну и ту же программу, то **будет создано два разных процесса**
- * Некоторые программы в процессе своей работы создают новые процессы
- * Например, Chrome создаёт **отдельный процесс для каждой вкладки**
- * Такой подход обеспечивает **лучшую безопасность за счёт изоляции процессов**, но повышает общее потребление памяти
- * **Процессы могут общаться друг с другом** с помощью механизмов ОС, сети или файловой системы

У КАЖДОГО ПРОЦЕССА СВОЯ ПАМЯТЬ

- * Свой стек, куча и другие сегменты
- * Хотя некоторые сегменты (например, .text) у однородных процессов могут разделяться, если ОС это поддерживает
- * ОС запрещает процессам вторгаться в память друг друга — вместо этого они должны явно общаться, используя доступные механизмы взаимодействия

ЗДЕСЬ РАЗНИЦА С ПОТОКАМИ

- * Процесс может **создать более одного потока исполнения**, чтобы распараллелить свою работу на нескольких ядрах процессора
- * Все потоки используют единую память процесса, но инициализируют свой **собственный сегмент стека, а вот куча у них общая**
- * В этом плане потоки **экономнее по памяти, но менее безопасны**

АДРЕСНОЕ ПРОСТРАНСТВО ДЛЯ ПРОЦЕССОВ

- * ОС выделяет адресное пространство памяти каждому процессу
- * Оно состоит из множества интервалов от N до M
для разных сегментов памяти программы: стек, куча, код и другие
- * Если программа не использует какой-либо сегмент (например, не работает с кучей), то ОС не выделяет под него пространство
- * ОС устанавливает стартовые значения для регистра инструкций и регистров стека
- * И загружает код программы по специальным адресам
(согласно формату исполняемого файла)

ПРОГРАММА И ОС

- * ОС поддерживает стандартный формат исполняемого файла — если программа ему не следует, она просто не запустится
- * Если мы пишем на ассемблере, то это отчасти **становится нашей задачей**
- * Когда мы пишем на высокоуровневом языке — это **задача компилятора**
- * Правильно сказать, что ОС просто очерчивает игровые рамки, а компилятор волен поступать как знает, но **не должен нарушать обозначенных границ**

ОТОБРАЖЕНИЕ ПАМЯТИ

- * Адресное пространство, которое выделяется процессу, **является виртуальным**
- * Это означает, что оно **не отображается напрямую на физические ячейки** памяти
- * ОС управляет **отображением виртуальной памяти на физическую**
- * Это ключевой момент **необходимый для многозадачности**
- * Программы используют адресное пространство, которое дала им ОС, как хотят, но **случайно пересечься они не могут**

КАЖДЫЙ ПРОЦЕСС ЖИВЁТ В СВОЕЙ «ПЕСОЧНИЦЕ»

- * Процесс А думает, что его память лежит по адресам 0x00000000 – 0xFFFFFFFF
- * Процесс Б тоже думает, что его память лежит по этим адресам
- * Но **физически их данные лежат в разных местах оперативной памяти**, а ОС с помощью таблиц страниц просто подменяет картинку

ВИРТУАЛЬНАЯ ПАМЯТЬ != ФИЗИЧЕСКАЯ

- * Тот факт, что ОС выделила программе адресное пространство, **не означает фактического выделения физической памяти**
- * Физическая память выделяется только тогда, **когда процесс начинает обращаться к ней** (читать или писать)
- * Может возникнуть ситуация, что формально виртуальной памяти хватает, но **реальной физической памяти уже не хватает**

КАК ТАКОЕ
МОЖЕТ
ПРОИЗОЙТИ?



ИЗ-ЗА ДРУГИХ ПРОГРАММ

- * В многозадачной ОС **одновременно может существовать множество процессов**
- * Суммарное использование физической памяти всеми процессами **может превысить её объём, что приведёт к глобальной нехватке**
- * В таком случае ОС может **выгрузить память программ**, которые сейчас не исполняются, на диск и отдать её активному процессу
- * Это называется **свопом** (или подкачкой страниц)
- * Когда такое происходит, производительность программы **может сильно упасть**

СТРАНИЧНАЯ АДРЕСАЦИЯ

- * Вся виртуальная память процесса разбита на маленькие блоки фиксированного размера — страницы (обычно 4 КБ)
- * Физическая память тоже разбита на блоки того же размера — кадры (frames)
- * ОС ведёт таблицы, которые говорят: **какая виртуальная страница какому физическому кадру** соответствует
- * Если соответствия нет — значит, страница либо не выделена, либо выгружена на диск

А ЧТО ЗНАЧИТ ПРОЦЕСС,
КОТОРЫЙ СЕЙЧАС
НЕ ИСПОЛНЯЕТСЯ?



ПОСЛЕДНЯЯ ЧАСТЬ ПАЗЛА

- * Процессы могут исполняться параллельно только в том случае, если они выполняются на отдельных ядрах процессора
- * В остальных случаях многозадачность достигается за счёт того, что ОС оркестрирует исполнение процессов через установку на паузу и возобновление
- * То есть процесс выполняется в течение небольшого кванта времени, затем ОС ставит его на паузу и передаёт процессорное время другому процессу
- * Когда ядер несколько, ОС распределяет нагрузку между ними

ВЫТЕСНЯЮЩАЯ МНОГОЗАДАЧНОСТЬ

- * Важнейшей частью ОС является планировщик процессов
- * Именно он решает, кому и на сколько выдать процессорное время
- * Планировщик может в любой момент прервать процесс, чтобы поставить его на паузу
- * Поэтому такая модель многозадачности называется вытесняющей

А КАКАЯ **ЕЩЕ** БЫВАЕТ
МНОГОЗАДАЧНОСТЬ?



КООПЕРАТИВНАЯ МНОГОЗАДАЧНОСТЬ

- * Главное отличие тут в том, что в такой модели тот, кто исполняется сам добровольно решает в какой момент прерваться
- * Планировщик в таком подходе только решает, кого запустить следующим
- * Такой подход активно используется в асинхронном программировании
- * У нас будет про это отдельная лекция

ФИНАЛЬНЫЙ АККОРД

- * ОС полностью **контролирует выполнение процессов**
- * Включая возможность **вежливо попросить процесс завершиться**
(например, через сигнал SIGTERM) – но процесс может эту просьбу проигнорировать
- * Или **принудительно завершить процесс**
(например, через SIGKILL или при нарушении)
- * Со смертью процесса все ресурсы, которые он занимал,
автоматически освобождаются

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Как мы можем видеть, **ОС делает огромную работу**, чтобы мы могли одновременно запустить у себя на устройстве несколько программ
- * С приходом многоядерных систем **сложность возросла многократно**
- * Так как в большинстве случаев мы разрабатываем ПО для работы под управлением ОС, нам **нужно понимать основы** того, как она работает
- * Здесь будет **полезно и интересно** почитать труды Э. Таненбаума

А КАК ПРОИСХОДИТ **УПРАВЛЕНИЕ** КУЧЕЙ В РАЗНЫХ ЯП?



ПО РАЗНОМУ

- * Ручное управление (ASM, C/C++)
- * Стековая модель (Algol)
- * Автоматические счетчики ссылок (Swift, Objective-C, Perl, Tcl)
- * Ручные счетчики ссылок (Rust, C++)
- * Гибриды: счетчики + детектор циклов (Python, PHP)
- * Трассирующие GC (поколенческие mark-sweep/compact/copy) (Go, Java, JS)
- * Статическое управление (Rust)

РАНЬШЕ

- * Раньше **процессоры работали в режиме реальных адресов** — это такой режим, в котором не было виртуальной памяти
- * Логические адреса **однозначно отображались на физические**
- * Например, в этом режиме работал DOS
- * При таком подходе не было многозадачности в современном понимании, а **программисты могли руками управлять памятью**

СЕЙЧАС

- * Современные процессоры работают в защищённом режиме (хотя режим реальных адресов оставлен для совместимости)
- * Программы в таком режиме работают с памятью, используя API операционной системы (например, аллокатор)
- * Это именно тот подход, который сейчас используется в низкоуровневом программировании
- * Даже программы на языках ассемблера сейчас в основном работают через API ОС

НЮАНСЫ РУЧНОГО УПРАВЛЕНИЯ

- * Полный контроль над освобождением ресурсов
- * Ошибиться очень просто
- * Ошибки имеют фатальные последствия
- * Легко сделать утечку памяти

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size = 5;

    // Выделяем память под массив из 5 целых чисел
    int *arr = (int*)malloc(size * sizeof(int));

    // Проверяем
    if (arr == NULL) {
        printf("Ошибка\n");
        return 1;
    }

    // Записываем значения
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;
    arr[3] = 40;
    arr[4] = 50;

    // Читаем
    printf("%d\n", arr[2]); // 30

    // Освобождаем
    free(arr);

    return 0;
}
```


НЮАНСЫ ПОДСЧЕТА ССЫЛОК

- * Возможная реализация **сборщика мусора**
- * **Небольшая цена** и простота реализации
- * **Проблемы**, если значение ссылается само на себя
- * **Легко сделать утечку памяти**

НАПРИМЕР, PERL

```

{
  my $a = {};    # хеш
  my $b = {};    # другой хеш

  $a->{other} = $b; # $a ссылается на $b
  $b->{other} = $a; # $b ссылается на $a — цикл замкнулся!
}
# Выход из блока: $a и $b выходят из области видимости
# Но их счётчики ссылок = 1 (они всё ещё ссылаются друг на друга)
# Память никогда не освободится — классическая утечка!
```

НЮАНСЫ ТРАССИРУЮЩИХ СБОРЩИКОВ

- * Большая цена в рантайме и сложность реализации
- * Необходимость делать «остановки» для очистки
- * Мы не управляем тем, когда память фактически очиститься
- * Легко сделать утечку памяти (но сложнее, чем со счетчиком)

НАПРИМЕР, JS

```
class Widget {  
  constructor() {  
    // Пока не удалим явно, весь экземпляр и замыкание будет висеть в памяти  
    document.addEventListener("click", this.onclick);  
  }  
  
  onclick() {  
    // ...  
  }  
}
```


А ЧЕМ СБОРЩИК НА
ССЫЛКАХ ОТЛИЧАЕТСЯ
ОТ ТРАССИРУЮЩЕГО?



РАЗНЫЕ РЕАЛИЗАЦИИ

- * В модели на счетчиках ссылок у каждого объекта в куче **есть специальный счетчик**
- * Когда **значение передается потребителю** – счетчик увеличивается
- * Когда **потребитель становится недоступен** – счетчик уменьшается
- * Когда **счетчик достиг 0** – происходит очистка памяти
- * В трассирующем варианте сборщик раз в некоторое время **останавливает программу и проходит по дереву ссылок**
- * Те узлы, которые **оказываются недостижимы при обходе** – помечаются для удаления

ПОКОЛЕНЧЕСКАЯ СБОРКА ПРИ ТРАССИРОВКЕ

- * Только что созданные объекты попадают в **молодое поколение**
- * GC проверяет их **достижимость часто** – большинство объектов умирает молодыми
- * Если объект пережил несколько сборок – **продвигается в старое поколение**
- * GC проверяет **достижимость в старом поколении куда реже**
- * Молодые и старые поколения **хранятся в разных областях кучи**
- * **Старые объекты** могут ссылаться на молодые

ИНКРЕМЕНТАЛЬНАЯ СБОРКА

- * Современные трассирующие GC **делают свою работу частями** — чтобы уменьшить паузы в исполнении кода
- * Но из-за этого **мусор может накапливаться** между фазами сборки: память уже не достижима для программы, но GC ещё не успел её освободить
- * В результате **потребление памяти растёт** выше теоретического минимума, и в моменты принудительной полной сборки всё равно возможны заметные паузы

КОПИРОВАНИЕ ПАМЯТИ

- * Современные трассирующие GC **используют стратегию копирования** для объектов из молодого поколения
- * Смысл в том, что проще не удалять мусорные узлы из старой области памяти, а **скопировать все живые узлы в новую чистую область**
- * Так как **сегмент памяти молодого поколения небольшой**, живых объектов мало – копировать их быстро
- * Дополнительные плюсы: **нет фрагментации памяти** и **не нужно проходить всю кучу** для sweep-фазы

ВЫВОДЫ

- * Счетчики ссылок проще в реализации и **детерминированно освобождают память** сразу, как только объект стал не нужен
- * Но в них **особенно просто допустить утечку памяти** из-за **циклических ссылок**
- * Трассирующие сборщики **не страдают проблемой циклов, но требуют больше ресурсов** и не дают гарантий, когда именно освободится память
- * **Существуют гибриды** – например, счетчики ссылок + детектор циклов или трассирующий GC, использующий счетчики для узких мест
- * Сборка мусора – **это реально сложно**

НЮАНСЫ RUST

- * **Скорость** как при ручном управлении
- * Можно использовать **ссылки на стек, а не только на кучу**
- * **Безопасность** как в языках со сборкой мусора
- * **Сложно сделать утечку памяти**
- * **Сложность самого ЯП**

НАПРИМЕР



```
fn main() {  
    let v1 = vec![1, 2, 3]; // v1 владеет вектором в куче  
    let v2 = v1;           // владение ПЕРЕХОДИТ к v2, v1 становится недействительным  
  
    println!("{:?}", v2); // ✅ работает — v2 владелец  
    println!("{:?}", v1); // ❌ ошибка компиляции: значение перемещено из v1  
}
```

// v2 выходит из области видимости — вектор автоматически освобождён

RAII

- * В Rust и C++ активно **используется концепция RAII** (Resource Acquisition Is Initialization)
- * Это означает, что мы **точно знаем, когда ресурс освободит свою память**
- * И **можем подвязаться на это поведение** – добавить деструктор

НАПРИМЕР

```
struct Logger;

impl Drop for Logger {
    fn drop(&mut self) {
        println!("Ресурс освобождён!");
    }
}

fn main() {
    let _log = Logger;
    let v = vec![1, 2, 3];
} // drop() вызовется здесь, детерминированно
```


second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ИДЕАЛЬНОГО РЕШЕНИЯ НЕТ

- ✱ Rust, казалось бы, совместил скорость, безопасность и надёжность — но **заплатил за это сложностью языка**
- ✱ Мы же с вами пишем на JavaScript, и здесь свои нюансы, которые **нужно понимать и учитывать**

ЕСЛИ НА ОБЪЕКТ НЕТ ДОСТИЖИМЫХ ССЫЛОК

- * Сборщик мусора может **поставить такой объект на уничтожение**
- * Проблема в том, что мы **не знаем, когда он это сделает**
- * Но можно **запустить сборку мусора явно** через отладчик или специальный API в Node.js (это полезно при отладке)
- * А также можно зарегистрировать колбек, который вызовется, **когда объект будет уничтожен сборщиком мусора**



```
// Создаём реестр: при уничтожении объекта вызывается колбэк
const registry = new FinalizationRegistry((heldValue) => {
  console.log(`🗑️ Объект "${heldValue}" уничтожен сборщиком мусора`);
});

let user = { name: "Андрей", age: 42 };

// Регистрируем объект, передаём "идентификатор" для колбэка
registry.register(user, user.name);

// Удаляем ссылку
user = null;

// Запускаем сборку мусора в Node.js
// (запускать с флагом --expose-gc)

global.gc();
```


ЗАЧЕМ ЭТО
НУЖНО?



ОТЛАДКА И ДЕСТРУКТОРЫ

- * Часто бывает нужно убедиться, что объект действительно уничтожается и его память освобождается – это помогает искать утечки памяти
- * Или **добавить логику деструктора**
- * Например, закрыть соединение с базой или файловой дескриптор
- * К сожалению, мы **не знаем, когда сборщик удалит объект**
- * Для деструкторов **есть вариант куда лучше**

```
import * as fs from "node:fs";

class File {
  fd;

  constructor(fd) {
    this.fd = fd;
  }

  [Symbol.dispose]() {
    fs.closeSync(this.fd);
  }
}

{
  using file = new File(fs.openSync("./example.txt", "w"));
  using file2 = new File(fs.openSync("./example2.txt", "r"));

  fs.writeFileSync(file.fd, "test");
  fs.writeFileSync(file2.fd, "test");
}
```

```
import * as fs from "node:fs/promises";

class File {
  handle;

  constructor(handle) {
    this.handle = handle;
  }

  async [Symbol.asyncDispose]() {
    await this.handle.close();
  }
}

(async () => {
  await using file = new File(await fs.open("./example.txt", "w"));
  await using file2 = new File(await fs.open("./example2.txt", "r"));

  await file.handle.write("test");
  await file2.handle.write("test");
})();
```

РЕФЛЕКСИРУЕМ

- * Благодаря новым символам `Symbol.dispose` и `Symbol.asyncDispose` мы можем добавить деструктор для любого объекта
- * Деструктор будет вызываться автоматически при использовании новых ключевых слов `using` и `await using`
- * `using` похож на `const`, но с привязкой времени жизни к области видимости
- * Как только переменная покидает свою область видимости – вызывается деструктор
- * Это позволяет реализовать RAII-подход в JS

ГДЕ ЭТО РАБОТАЕТ?



ПОДДЕРЖКА НА УРОВНЕ

- * В Node.js это уже давно работает нативно
- * В браузере поддержка делается через TS или другие транспайлеры

СЛАБЫЕ ССЫЛКИ

- * В JS есть специальные виды контейнеров, которые позволяют держать ссылку на объект, **но не препятствуют его удалению**, если других ссылок на него нет
- * Это бывает очень полезно для организации автоматически очищаемого кэша или других похожих сценариев



```
// Кэш метаданных, который не мешает GC
const userCache = new WeakMap();

let user = { id: 1, name: "Alice" };

userCache.set(user, { lastLogin: "2026-05-14" });

console.log(userCache.get(user)); // { lastLogin: "2026-05-14" }

user = null; // других ссылок на объект нет

// Объект будет удалён GC, метаданные из WeakMap исчезнут автоматически
```

```
const visited = new WeakSet();

function process(item) {
  if (visited.has(item)) {
    console.log("Уже обработано, пропускаем");
    return;
  }

  console.log("Обрабатываю:", item);
  visited.add(item);
}

let obj1 = { data: 1 };
let obj2 = { data: 2 };

process(obj1); // Обрабатываю: { data: 1 }
process(obj1); // Уже обработано, пропускаем
process(obj2); // Обрабатываю: { data: 2 }

obj1 = null; // GC может удалить obj1, visited не держит его живым
```

```
// Создаём объект
let data = { hugeArray: new Array(1000).fill("x") };

// Слабая ссылка на него
const ref = new WeakRef(data);

console.log(ref.deref()); // { hugeArray: [...] } - объект жив

data = null; // убираем сильную ссылку

// GC запускается (гипотетически)
setTimeout(() => {
  const maybeAlive = ref.deref();

  if (maybeAlive) {
    console.log("Объект ещё жив");
  } else {
    console.log("Объект уже собран GC");
  }
}, 1000);
```


second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

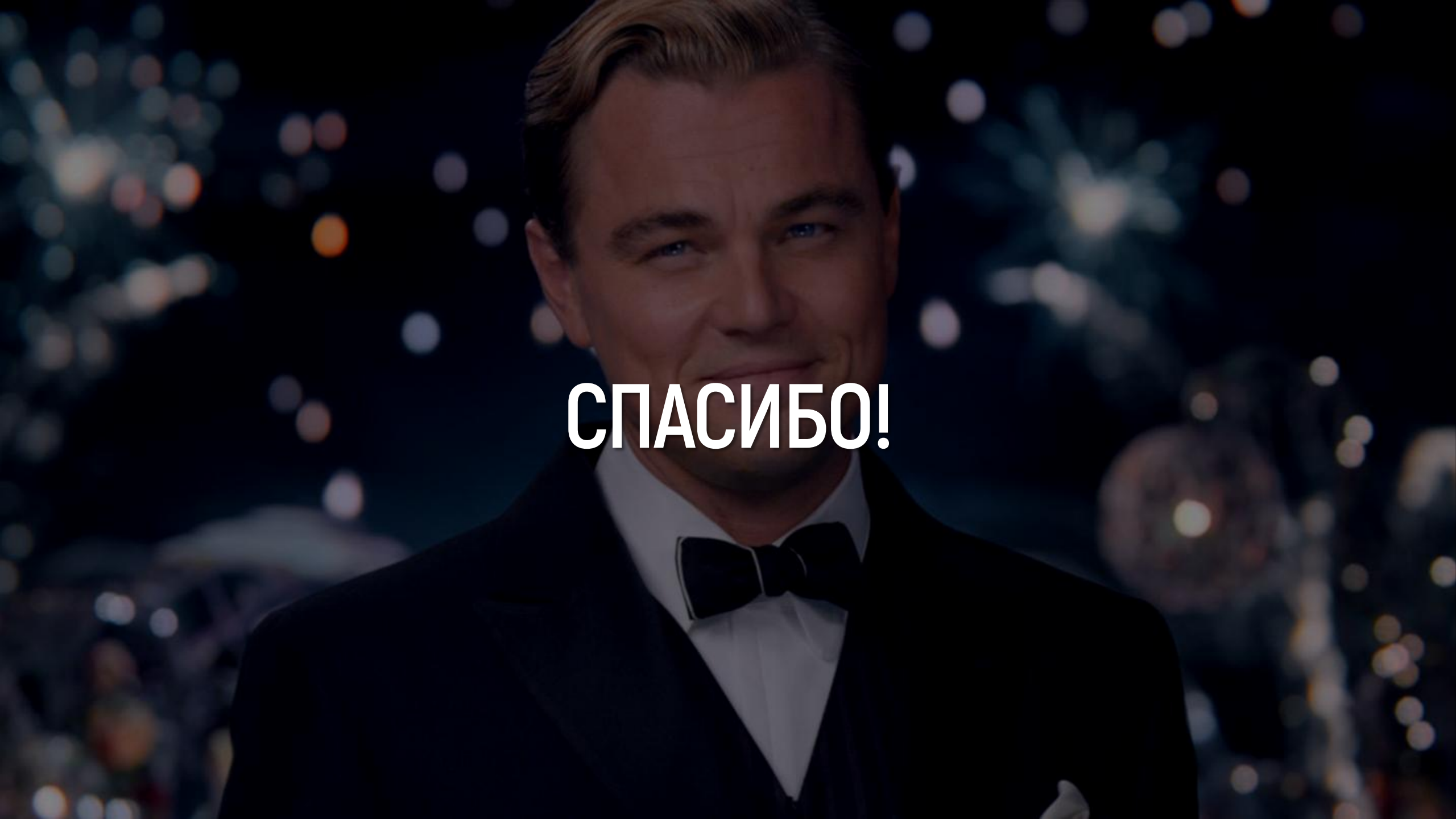
СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- ✱ Наличие сборщика мусора в JS **не мешает легко создавать утечку памяти**
- ✱ Грамотно используя слабые ссылки, мы можем создать архитектуру, **устойчивую к таким ошибкам**

ПОДВЕДЕМ ИТОГИ

- * ОС проделывает огромную работу, чтобы мы могли одновременно чатиться в мессенджере, слушать музыку и запускать локальную модельку
- * Многозадачность достигается за счет концепции виртуальной памяти, процессов и планировщика задач
- * Если ОС вводит базовые рамки работы с памятью, то каждый язык программирования **может по-своему вписываться в них**
- * В JS используется трассирующая сборка мусора, поэтому мы **достоверно не знаем, когда именно будет удалён наш объект**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!